

HPCA Assignment 1

NAME: KALIDINDI SRIDHAR VARMA

S.R. No: 28574

Course: M.Tech (course work), CSE

Top 3 Learnings:

Learning 1: The Algo

Firstly I would like to talk about the merits of this hawkeye policy which I read in the pinged paper. The authors had given an excellent method to come up with what assumed to be impossible prior atleast as per my knowledge. They presented Hawkeye as near belady's algorithm via learning the patterns from the history. They basically segmented the components of the algorithm into 3 major parts.

1. OPTGen algorithm whose job is to replicate the OPT algo. It starts with maintaining the state occupancy vector one per set whose size is $8 \times$ the associativity of the set. Each entry's max value is associativity-1. The idea is that when a new cache line is taken up the decision as to whether this is cache friendly or not is deferred to next access to that cache line. The time between the first access and immediate next access [can be interpreted as consecutive access to the same cache line] is termed as usage interval. Another term that was mentioned is liveliness interval which refers to the amount of time the cache line resides in the memory from in to eviction of it. The logic is that if more cache lines than the associativity of the set contend to occupy a set in the usage interval of a particular cache line then that line is evicted with no option left. That is if the number of overlapping liveliness intervals exceeds the set associativity in the span of usage intervals of a particular line that results in cache miss of that line in subsequent access to that line. The occupancy vector is updated and maintained accordingly starting with the most recent entry of the occupancy vector is set to 0 (say for line X). When line X is loaded for the first time, no further changes to the occupancy vector are made, reflecting the fact that OPT makes decisions based on reuse of that line. If X is not a first-time load, OPTgen checks to see if every element corresponding to the usage interval is less than the cache capacity: if so, then OPT would have placed X in the cache, so the part of occupancy vector that starts from previous access of X till the current access time stamp(excluding current time stamp) is updated as +1; if not, then X would have been a cache miss, so the occupancy vector is not modified. Also in this module we will maintain a sample cache which basically stores the cache copy imitating exactly how cache entries will look if we use OPT algo alone, in paper it is mentioned as sample cache as it is discussed after set dueling so I too referred to it as so but the purpose is same.

2. Predictor whose job is given PC of the instruction which made request for particular cache line this module has to emit either the incoming line is `CACHE_FRIENDLY/CACHE_AVERSE` this is done via maintaining a 8K entry indexed using 13-bit PC hash and each entry of size 3 bit maintained as saturating counter [0-7]. This predictor is trained by OPTGen algo that was discussed in section 3.1 of paper as if a particular line is interpreted as cache hit in OPTGen then that last PC that accesses this

cache line counter corresponding to that pc is increment in case of cache miss that is decremented.

3. The third module is the actual replacement algorithm itself [The Hawkeye] that basically maintains a 3 bit saturating counter per cache line. For the incoming cache line we get the prediction from the predictor module and set the RRIP value according to table 1 in the paper. [I don't want to put that entire discussion on combinations of miss/hit vs prediction here as it is exact detail mentioned in paper in single long para very precise and dense]. When the time to evict a line from a set we choose the line with RRIP value=7, if no such line exists choose line with highest rrip value as victim and then increment all RRIP values of all lines in the set by 1 [only lines with rrip<6 and currently predicted as cache_friendly, as mentioned in paper]. Here an interesting detail that was missed in the code templated in the assignment is detrain function which basically decrements the counter of that respective PC of the cache line that actually predicts as cache friendly and is in optgen sampler cache but still gets evicted. The algo logically designed to behave like LRU in phase-change between working sets and in-phase follows the above description.

Learning 2: The practical caveat mentioned in the paper.

The part of the paper that we are not asked to implement in section 3.2 introduces the motivation why exactly this direct implementation of this well structured algorithm yields no surprising results logically due to extreme overhead of memory maintained to run each module. So they came up with the idea of granularity of the occupancy vector which now allocated each entry to a time stamp of 4 units rather than 1. This makes the per set occupancy vector size to decrease size by a factor of 4. Next is set dueling , simply put we randomly choose only 64 sets out of all sets in the cache and run our algo on that set to make decisions for the rest of the cache.

Learning 3: Working around with ChampSim

While trying to write the adapter (hawkeye.cc) to test the hawkeye on traces it is really interesting that the ChampSim is quite simple to use. It supports many functionalities of a cache and in fact when probed it also includes utilities to work around with multi-core systems maintaining common cache at higher levels although I did not use those parameters in my code. Also this is my first time working around with traces. ChampSim became quite handy as a bash file could streamline the entire run of trace. Also ChampSIM has a very neat configuration file which has all the necessary tweakings we can do and that's it it has a bash file that takes this configuration and generates the make file itself without user intervention. The balance between tweaking allowed and simplicity to run them is quite impressive.

Observation from experiments [LRU vs Hawkeye]:

Observation 1 [Practical missing caveats]: Firstly if the code is implemented just as described in the paper it yielded worse results [I literally got negative improvement over LRU for some benchmarks]. There are subtle important points to be included into code which

authors did not mention in the paper but they had a public repository that has hawkeye implementation. I used that as a reference to learn some caveats. Those are as follows.

1. When a particular cache line is loaded into LLC. It is not always the case that it is due to load or auto increment of PC in which fetch unit calls brings next instruction [In both of these there exists a calling pc]. But for the lines which were loaded into LLC due to eviction of those lines from higher cache has no calling pc and paper mentioned nothing about what to do with these lines. In champsim they arrived into LLC with their ip=calling_set set to 0. I took the required workover from the authors repo as they classified this as CACHE_AVERSE directly without probing the predictor [we can't ask the predictor for prediction for these lines as the predictor is indexed using calling_pc of that line]. Previously I didn't realise this and got bad results for the benchmark because probing the predictor with pc=0 for all these lines would lead to bad results.
2. As mentioned in the paper there is a phase called detrain [mentioned in section 3.4]. As the assignment said to be evaluated on these 3 modules independently but to detrain there should exist a module which has predictor object, optgen object and rrip object together in some module which is not the case. So I did not implement that. In hawkeye adapter to simulate benchmarks I did not use this thing because I am completely lost in figuring out the above mentioned point 1 and without that detrain worsen the already worst results.

Observation 2:

As mentioned in the paper the hawkeye will perform better as set associativity increases. This is reflected in the plot 1 where though marginally LRU is better at associativity=4 hawkeye outperforms it at associativity=16 as associativity increases.

Observation 3:

The exact percentages won't match the paper. Because this is not a complete implementation the set duality not only reduces memory footprint but also reduces overall noise which I did not implement in assignment as it is mentioned not to. Also detrain I did not implement for the same reason and due to over experimentation as pointed out in observation 1. Also in paper they did 50M inst warmup and 200M inst testing we used much lesser numbers 20M and 50M respectively hence I think we gave lesser time for algo to settle down into steady state. Also I may have missed some practical caveats like mentioned in observation 1. But still I am able to reproduce the trends as mentioned in the paper in experiments.

NOTE: As mentioned in the teams rrpv values are on cache miss if no line with rrpv=7 was found we increment all other cache lines rrpv till some line reaches 7 and select it as victim. This is not inline with what is mentioned in the paper. In paper they mentioned evicting the cache line with highest RRPV value and specifically said cache friendly lines can never saturate to 7. This particular tweak although looks small I got better results with the first one with values as follows (hmmer +2.48% against +1.60%, with mcf and astar essentially unchanged). But I reported values that reflect the current code in my repo.

THANK YOU